

## Practical No. 1

```
def dfs(graph, node, visited):
    visited.add(node)
    print(node, end=' ')
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)
```

```
def bfs(graph, start):
    visited = set([start])
    queue = [start]
    while queue:
        node = queue.pop(0)
        print(node, end=' ')
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```

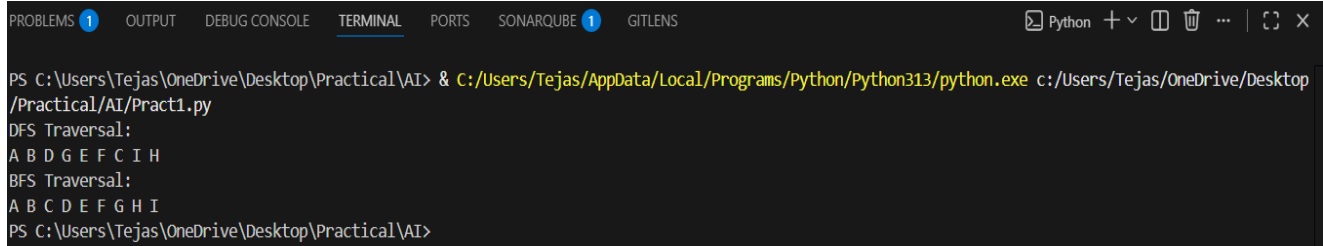
# Larger Graph representation

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B', 'G'],
    'E': ['B', 'F', 'H'],
    'F': ['C', 'E', 'I'],
    'G': ['D'],
    'H': ['E'],
    'I': ['F']
}
```

```
print("DFS Traversal:")
dfs(graph, 'A', set())
```

```
print("\nBFS Traversal:")
bfs(graph, 'A')
```

## OUTPUT :-



The screenshot shows a Visual Studio Code terminal window with the following content:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 1 GITLENS Python + v [icon] [icon] ... [icon] [icon] X
```

PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop/Practical/AI/Pract1.py

DFS Traversal:  
A B D G E F C I H

BFS Traversal:  
A B C D E F G H I

PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>

## Practical No.2

```
def astar(graph, heuristic, start, goal):
    open_list = [(start, 0)]
    closed_list = set()
    parent = {start: None}
    g_cost = {start: 0}

    while open_list:
        # Sort based on  $f(n) = g(n) + h(n)$ 
        open_list.sort(key=lambda x: g_cost[x[0]] + heuristic[x[0]])
        current = open_list.pop(0)[0]

        if current == goal:
            break

        closed_list.add(current)

        for neighbor, cost in graph[current]:
            if neighbor in closed_list:
                continue

            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                parent[neighbor] = current
                open_list.append((neighbor, new_cost))

    # Reconstruct path
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]

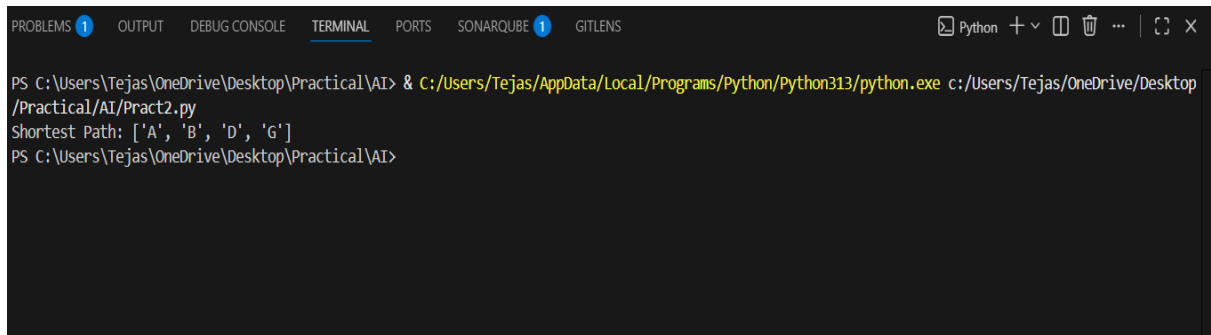
    return path[::-1]

# ----- Graph Definition -----
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

# ----- Heuristic Values -----
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
```

```
'D': 2,  
'E': 1,  
'F': 2,  
'G': 0  
}  
  
# ----- Run A* -----  
path = astar(graph, heuristic, 'A', 'G')  
print("Shortest Path:", path)
```

## OUTPUT :-



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 1 GITLENS  
Python + - [ ] [ ] ... | [ ] [ ] X  
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop  
/Practical/AI/Pract2.py  
Shortest Path: ['A', 'B', 'D', 'G']  
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>
```

## Practical No. 3

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap after inner loop finishes
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

# ----- Multiple Test Cases -----
test_cases = [
    [64, 25, 12, 22, 11],      # Normal case
    [5, 4, 3, 2, 1],          # Reverse order
    [1, 2, 3, 4, 5],          # Already sorted
    [3, 3, 2, 1, 2],          # Duplicates
    [-5, -1, -3, 2, 0],       # Negative numbers
    [10],                     # Single element
    []                        # Empty list
]

for i, data in enumerate(test_cases, 1):
    print(f"Test Case {i}:")
    print("Original:", data)
    print("Sorted  :", selection_sort(data.copy()))
    print("-" * 30)
```

**OUTPUT :-**

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 1 GITLENS
Python + v [Icons] X

PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop/Practical/AI/Practical/AI/Pract3.py

Test Case 1:
Original: [64, 25, 12, 22, 11]
Sorted : [11, 12, 22, 25, 64]
-----

Test Case 2:
Original: [5, 4, 3, 2, 1]
Sorted : [1, 2, 3, 4, 5]
-----

Test Case 3:
Original: [1, 2, 3, 4, 5]
Sorted : [1, 2, 3, 4, 5]
-----

Test Case 4:
Original: [3, 3, 2, 1, 2]
Sorted : [1, 2, 2, 3, 3]
-----

Test Case 5:
Original: [-5, -1, -3, 2, 0]
Sorted : [-5, -3, -1, 0, 2]
-----

Test Case 6:
Original: [10]
Sorted : [10]
-----

Test Case 7:
Original: []
Sorted : []
-----

PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>

```

## Practical No.4

```
def solve_n_queens(n):
    board = [-1] * n

    def is_safe(row, col):
        for i in range(row):
            # Check column and diagonal conflicts
            if board[i] == col or abs(board[i] - col) == row - i:
                return False
        return True

    def solve(row):
        if row == n:
            print("Solution:", board)
            return

        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                solve(row + 1)
                board[row] = -1 # Backtrack

    solve(0)

# ----- Multiple Test Cases -----
test_values = [4, 5, 6]

for n in test_values:
    print(f"\nSolving {n}-Queens Problem:")
    solve_n_queens(n)
    print("-" * 40)
```

## OUTPUT :-

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 1 GITLENS Python + - [ ] [ ] ... [ ] [ ] X
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop/Practical/AI/Pract4.py

Solving 4-Queens Problem:
Solution: [1, 3, 0, 2]
Solution: [2, 0, 3, 1]
-----

Solving 5-Queens Problem:
Solution: [0, 2, 4, 1, 3]
Solution: [0, 3, 1, 4, 2]
Solution: [1, 3, 0, 2, 4]
Solution: [1, 4, 2, 0, 3]
Solution: [2, 0, 3, 1, 4]
Solution: [2, 4, 1, 3, 0]
Solution: [3, 0, 2, 4, 1]
Solution: [3, 1, 4, 2, 0]
Solution: [4, 1, 3, 0, 2]
Solution: [4, 2, 0, 3, 1]
-----

Solving 6-Queens Problem:
Solution: [1, 3, 5, 0, 2, 4]
Solution: [2, 5, 1, 4, 0, 3]
Solution: [3, 0, 4, 1, 5, 2]
Solution: [4, 2, 0, 5, 3, 1]
-----

PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>
```



## Practical No. 5

```
def chatbot():
    print("Chatbot: Hello! How can I help you? (Type 'bye' to exit)")

    while True:
        user_input = input("User: ").lower()

        if "hi" in user_input or "hello" in user_input:
            print("Chatbot: Hello! How may I assist you?")

        elif "how are you" in user_input:
            print("Chatbot: I'm just a program, but I'm doing great! 😊")

        elif "name" in user_input:
            print("Chatbot: I am a simple Python chatbot.")

        elif "price" in user_input or "cost" in user_input:
            print("Chatbot: Please visit our website for pricing details.")

        elif "contact" in user_input or "support" in user_input:
            print("Chatbot: You can contact us at support@example.com")

        elif "services" in user_input:
            print("Chatbot: We provide AI, Web Development, and Data Science services.")

        elif "time" in user_input:
            from datetime import datetime
            now = datetime.now()
            print("Chatbot: Current time is", now.strftime("%H:%M:%S"))

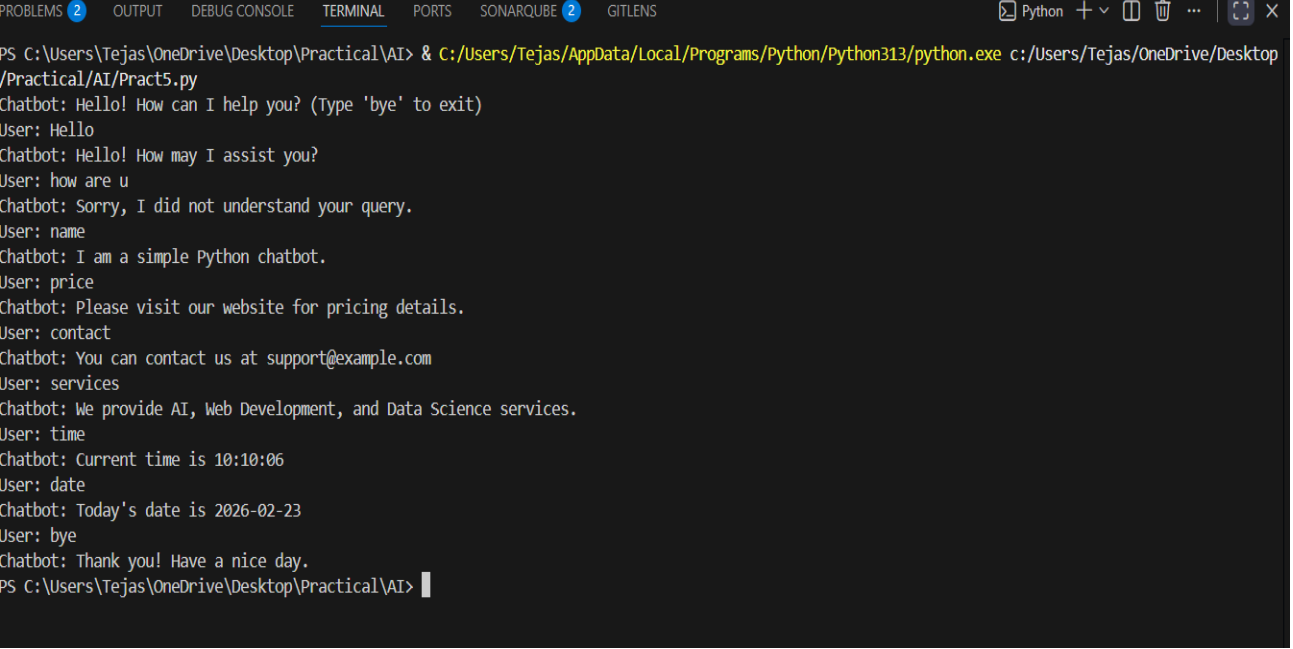
        elif "date" in user_input:
            from datetime import datetime
            today = datetime.now()
            print("Chatbot: Today's date is", today.strftime("%Y-%m-%d"))

        elif "bye" in user_input or "exit" in user_input:
            print("Chatbot: Thank you! Have a nice day.")
            break

        else:
            print("Chatbot: Sorry, I did not understand your query.")

# Run chatbot
chatbot()
```

## OUTPUT :-



The screenshot shows a Visual Studio Code interface with a terminal window open. The terminal title bar includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, SONARQUBE, and GITLENS. The terminal content shows a command prompt running a Python script. The script is a simple chatbot that responds to various user inputs. The chatbot's responses include a greeting, a request for help, a response to 'Hello', a response to 'how are u', an apology for not understanding, a response to 'name', a self-introduction, a response to 'price', a website recommendation, a response to 'contact', an email address, a response to 'services', a list of services, a response to 'time', the current time, a response to 'date', the current date, and a farewell message. The terminal ends with the command prompt prompt.

```
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop/Practical/AI/Pract5.py
Chatbot: Hello! How can I help you? (Type 'bye' to exit)
User: Hello
Chatbot: Hello! How may I assist you?
User: how are u
Chatbot: Sorry, I did not understand your query.
User: name
Chatbot: I am a simple Python chatbot.
User: price
Chatbot: Please visit our website for pricing details.
User: contact
Chatbot: You can contact us at support@example.com
User: services
Chatbot: We provide AI, Web Development, and Data Science services.
User: time
Chatbot: Current time is 10:10:06
User: date
Chatbot: Today's date is 2026-02-23
User: bye
Chatbot: Thank you! Have a nice day.
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>
```

## Practical No. 6

```
def hospital_expert_system():
    print("Hospital Expert System")
    print("Available symptoms:")
    print("1. fever")
    print("2. cough")
    print("3. chest pain")
    print("4. headache")
    print("5. stomach pain")
    print("6. skin rash")
    print("7. breathing problem")
    print("8. dizziness")
    print("9. weakness")
    print("\nType 'exit' to quit.")

while True:
    symptom = input("\nEnter your symptom: ").lower()

    if symptom == "exit":
        print("Thank you! Stay healthy ")
        break

    elif symptom == "fever":
        print("Advice: Consult General Physician.")

    elif symptom == "cough":
        print("Advice: Visit Pulmonology Department.")

    elif symptom == "chest pain":
        print("Advice: Consult Cardiology Department immediately.")

    elif symptom == "headache":
        print("Advice: Visit Neurology Department.")

    elif symptom == "stomach pain":
        print("Advice: Visit Gastroenterology Department.")

    elif symptom == "skin rash":
        print("Advice: Consult Dermatology Department.")

    elif symptom == "breathing problem":
        print("Advice: Visit Emergency or Pulmonology immediately.")

    elif symptom == "dizziness":
        print("Advice: Consult Neurology or General Physician.")

    elif symptom == "weakness":
        print("Advice: Get blood test and consult General Physician.")

    else:
        print("Advice: Symptom not recognized. Please consult General Physician.")
```

```
# Run the expert system
hospital_expert_system()
```

**OUTPUT :-**

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 2 GITLENS
Python + v [icons] ... | [icons] x
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI> & C:/Users/Tejas/AppData/Local/Programs/Python/Python313/python.exe c:/Users/Tejas/OneDrive/Desktop/Practical/AI/Pract6.py
📄 Hospital Expert System
Available symptoms:
1. fever
2. cough
3. chest pain
4. headache
5. stomach pain
6. skin rash
7. breathing problem
8. dizziness
9. weakness

Type 'exit' to quit.

Enter your symptom: 1
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 2
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 5
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 6
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 3
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 4
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: 8
Advice: Symptom not recognized. Please consult General Physician.

Enter your symptom: exit
Thank you! Stay healthy 😊
PS C:\Users\Tejas\OneDrive\Desktop\Practical\AI>

```